॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 15

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	**Raghav Sharma**
PRN No.	**23070122171**
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	23rd July, 2026
Title of Assignment:	Basic Infrastructure as Code (IaC) with Terraform
Theory:	Infrastructure as Code (IaC) is the practice of managing and provisioning infrastructure using configuration files instead of manual processes. Terraform is an open-source IaC tool developed by HashiCorp that enables users to define, provision, modify, and destroy infrastructure resources in a consistent and automated manner. In this experiment, Terraform was used with the Local provider to create, update, and destroy a local text file. The commands `terraform init`, `terraform plan`, `terraform apply`, and `terraform destroy` demonstrated Terraform's workflow of initializing the working directory, previewing infrastructure changes, applying the desired configuration, and safely removing managed resources. This approach ensures repeatable deployments, version control of infrastructure, and simplified infrastructure management. **Commands Used** • `terraform --version` • `mkdir my-terraform-iac`

	• `cd my-terraform-iac` • `terraform init` • `terraform plan` • `terraform apply` • `terraform destroy`

Implementation:	**Step 1: Install Terraform**
	`C:\Users\riavi>terraform --version`
	`Terraform v1.15.8`
	`on windows_amd64`
	Installed Terraform — Successfully installed Terraform on the Windows system and verified the installation using the `terraform --version` command.
	Step 2: Create the Project Directory
	`C:\Users\riavi>mkdir my-terraform-iac`
	`C:\Users\riavi>cd my-terraform-iac`
	Created Terraform Project Directory — Created a new project directory named `my-terraform-iac` and navigated into it to store the Terraform configuration files.
	Step 3: Create the Terraform Configuration File (main.tf)
	main.tf
	`terraform {`
	`  required_providers {`
	`  local = {`
	`  source  = "hashicorp/local"`
	`  version = "2.4.0"`
	`  }`
	`  }`
	`}`
	`provider "local" {`
	`}`
	`resource "local_file" "my_example_file" {`
	`  filename = "example.txt"`
	`  content  = "Hello, Terraform IaC!"`
	`}`

Created Terraform Configuration File — Created the `main.tf` configuration file and defined the Local provider along with a `local_file` resource to create a text file using Terraform.

Step 4: Initialize Terraform

```
C:\Users\riavi\my-terraform-iac>terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/local versions matching "2.4.0"...
- Installing hashicorp/local v2.4.0...
- Installed hashicorp/local v2.4.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Initialized Terraform — Initialized the Terraform working directory using the `terraform init` command, which downloaded the required provider plugins and prepared the project for execution.

Step 5: Plan the Changes

```
C:\Users\riavi\my-terraform-iac>terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

  # local_file.my_example_file will be created
  + resource "local_file" "my_example_file" {
      + content              = "Hello, Terraform IaC!"
      + content_base64sha256 = (known after apply)
      + content_base64sha512 = (known after apply)
      + content_md5          = (known after apply)
      + content_sha1         = (known after apply)
      + content_sha256       = (known after apply)
      + content_sha512       = (known after apply)
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "example.txt"
      + id                   = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if
you run "terraform apply" now.

C:\Users\riavi\my-terraform-iac>
```

Generated Terraform Execution Plan — Generated an execution plan using the `terraform plan` command to preview the infrastructure changes before applying them. The plan indicated that one new resource (`local_file.my_example_file`) would be created with no modifications or deletions.

Step 6: Apply the Changes

```
Terraform used the selected providers to generate the following execution p
lan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # local_file.my_example_file will be created
  + resource "local_file" "my_example_file" {
      + content              = "Hello, Terraform IaC!"
      + content_base64sha256 = (known after apply)
      + content_base64sha512 = (known after apply)
      + content_md5          = (known after apply)
      + content_sha1         = (known after apply)
      + content_sha256       = (known after apply)
      + content_sha512       = (known after apply)
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "example.txt"
      + id                   = (known after apply)
    }

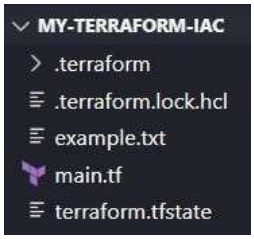
Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.my_example_file: Creating...
local_file.my_example_file: Creation complete after 0s [id=67c458d620445706
ef3f2d7483fd5f652f7ba55b]

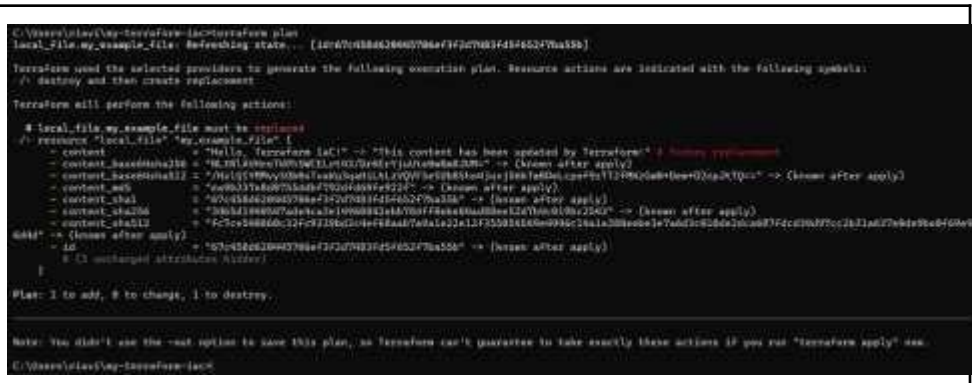
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```



Applied Terraform Configuration — Applied the Terraform configuration using the `terraform apply` command, which successfully created the `example.txt` file as defined in the `main.tf` configuration. The generated file was verified in the project directory.

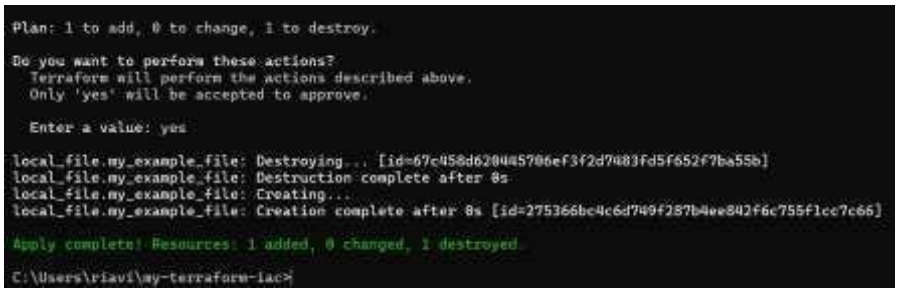
Step 7: Modify the Content and Re-apply

Command: `terraform plan`



Generated Updated Execution Plan — Modified the `main.tf` configuration and generated an updated execution plan using the `terraform plan` command. Terraform detected that the existing `local_file` resource would be replaced to apply the updated content.

Command: `terraform apply`



Applied Updated Terraform Configuration — Applied the modified Terraform configuration using the `terraform apply` command. Terraform successfully recreated the `local_file` resource with the updated content.

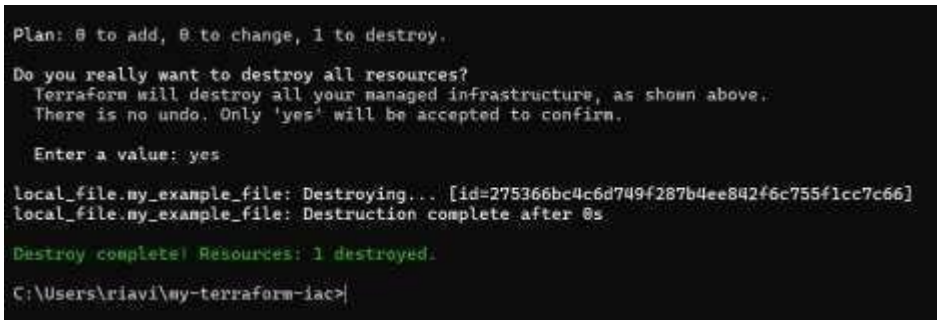
Verified Updated File Content — Verified that the `example.txt` file was successfully updated with the new content after applying the modified Terraform configuration.

Step 8: Destroy the Resource

Command: `terraform destroy`



Destroyed Terraform Resource — Executed the `terraform destroy` command, confirmed the destruction by entering `yes`, and successfully

	removed the Terraform-managed `local_file` resource from the local system.  Verified Resource Deletion — Verified that the `example.txt` file was successfully removed from the project directory after the Terraform-managed resource was destroyed.

Subject Faculty Name: Prof. Arijit Dutta